

— COMP4052 • FINAL PROJECT

MimbleWimble Confidential Transactions

Hiding transaction amounts on a blockchain while still proving no value is created — verified on-chain in Solidity, cross-validated against the Grin reference implementation.

Ege Deniz • 220304118 • Introduction to Blockchain & DLT

THE PROBLEM

Public blockchains expose every amount.

On Bitcoin, anyone can see who paid whom and how much. Privacy-preserving chains hide this in different ways:

Monero

Ring signatures

Hides sender, receiver, and amount via ring signatures, stealth addresses, and RingCT.

Zcash

zk-SNARKs

Hides everything with zero-knowledge proofs — at the cost of a trusted setup.

MimbleWimble

Commitments

Hides amounts with homomorphic Pedersen commitments. No addresses, no trusted setup.

THE SOLUTION

— IN ONE SENTENCE

A blockchain where
amounts are **hidden**, yet
every transaction stays
mathematically verifiable.

Proposed anonymously in 2016; used in production by Grin and Beam.

— THE PROTOCOL

Four cryptographic moves.

01 · Hide

Pedersen commitment

A commitment locks an amount behind a random blinding factor. Hiding, and binding.

$$C = v \cdot H + r \cdot G$$

02 · Check

Balance the books

Commitments are homomorphic, so the network verifies balance without seeing amounts.

$$\Sigma C_{in} - \Sigma C_{out} - fee \cdot H = excess \cdot G$$

03 · Prove

Kernel signature

A Schnorr signature on the excess proves balance and ownership — no value created.

$$s \cdot G = R + e \cdot P$$

04 · Protect

Range proof

Proves every amount is non-negative, defeating the negative-amount inflation attack.

$$0 \leq v < 2^{64}$$

— ARCHITECTURE

An on-chain verifier, a prover, and a reference.

Solidity

On-chain verifier

Verifies the kernel signature and commitment balance on Ethereum, with secp256k1 implemented from scratch.

MIN SOLUTION

Python

The prover

Builds confidential transactions — commitments, range proofs, kernel signatures — and verifies them off-chain.

Grin

The reference

grinventions/mimblewimble-py. Identical inputs through it and this project must agree.

THE PROPOSED SOLUTION

Verifying confidential transactions on Ethereum.

- ◆ **verifyKernelSignature** — confirms the spender knows the kernel excess (proves ownership).
- ◆ **verifyBalance** — confirms $\Sigma \text{ inputs} - \Sigma \text{ outputs} - \text{fee} \cdot H$ equals the kernel excess (no inflation).
- ◆ **secp256k1 from scratch** — point add, double, scalar-mul, and modular inverse, since the EVM has no curve precompile.
- ◆ **Range proofs stay off-chain** — on-chain they would cost tens of millions of gas; verified in the Python prover instead.

— INSIDE THE VERIFIER

The no-inflation check.

Sum the input commitments, subtract the outputs and the fee term, and confirm the result equals the published kernel excess. If the amounts did not balance, an amount term would remain and the check would fail.

```
uint256 x = inX[0]; uint256 y = inY[0];  
for (i=1; i<inX.length; i++)           // add inputs  
    (x,y) = ecAdd(x,y, inX[i],inY[i]);  
for (i=0; i<outX.length; i++)           // subtract outputs  
    (x,y) = ecAdd(x,y, outX[i],(PP-outY[i])%PP);  
if (fee  $\neq$  0) {                           // subtract fee·H  
    (fx,fy) = ecMul(HX,HY,(NN-fee)%NN);  
    (x,y)    = ecAdd(x,y, fx,fy);  
}  
return x==kernelExcessX && y==kernelExcessY;
```

 VERIFICATION

Cross-validated against Grin, and fully tested.

```
● ● ●  
  
# reference vs this implementation, 5 scenarios  
$ python scripts/compare_with_reference.py  
  Result: ALL SCENARIOS AGREE (5 / 5)  
  
$ pytest tests/      → 21 passed  
$ npx hardhat test   → 10 passing
```

SCOPE & FUTURE WORK

Known limitations.

- ◆ **Cut-through & kernel aggregation** — not implemented; this is the CT primitive layer.
- ◆ **Bulletproofs** — would replace the classical range proof and shrink it roughly ten-fold.
- ◆ **Gas optimization** — affine coordinates kept for clarity; Jacobian + wNAF would cut on-chain cost.
- ◆ **No P2P layer** — the focus is the cryptographic core, not networking.

REFERENCES

Foundations.

[Jedusor \(2016\)](#) – the original MimbleWimble text.

[Poelstra \(2016\)](#) – the canonical MimbleWimble write-up.

[Maxwell](#) – Confidential Transactions.

[Grin documentation](#) – docs.grin.mw

[grinventions/mimblewimble-py](#) – reference implementation.

MimbleWimble Confidential Transactions · [Ege Deniz](#) · COMP4052